Hard Negative Mining for Knowledge Graph Link Prediction: A Controlled Ablation on FB15k-237

Thalia Delhaise Lenny Briclet André Fonseca Rafael Gufler
RC2526, Knowledge Graphs, Deliverable 3 (Final Report)
Joint project with Deep Learning (Project 425282 – Milestone 2)

Abstract

*We study link prediction on **FB15k-237** using* TRANSE *and* ROTATE *as matched baselines, then investigate whether the choice of negative-triple selection strategy during training matters. Our main contribution is a **two-stage negative sampling pipeline** implemented in standalone PyTorch: Stage 1 generates a filtered pool of $n = 64$ Bernoulli-corrupted candidates (no known-true triple included); Stage 2 selects $k = 8$ training negatives using one of three strategies:* random*,* hard *(top-k by current model score), or* mixed *(Binomial split). Across five controlled runs with all other hyperparameters fixed, **hard-negative selection raises MRR from 0.270 to 0.298 (+10.4%), with the best mixed variant (70% hard) reaching 0.299**; a 30% hard mixture already recovers 95% of that gain. Relation-wise slice evaluation shows the largest improvements on* rare *relations (+3.6 pp) and* high-degree *tail entities (+2.9 pp). Qualitative examples confirm that hard-negative training reduces the rank of the correct entity from 9–17 (random) down to 1–6 on structured relation types.*

1. Introduction

A **knowledge graph** (KG) is a directed multi-relational graph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ where each triple $(h, r, t) \in \mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ encodes a binary fact. **Link prediction** (KG completion) asks: given a partial query $(h, r, ?)$ or $(?, r, t)$, rank all candidate entities.

Training KG embedding (KGE) models requires *negative* triples, facts that are false. Standard Bernoulli corruption [5] replaces the head or tail uniformly at random. After a few training epochs the model has already learned to reject trivially implausible negatives, so the gradient signal from easy negatives becomes small and uninformative. Hard negatives, false triples that the current model still scores highly, carry more information per update, but training exclusively on them can be unstable.

Research question. For link prediction on FB15k-237, does replacing uniform random negatives with model-scored hard negatives improve test-set MRR and Hits@k? Does a mixture of random and hard negatives yield a better accuracy-stability trade-off?

Contributions.

- Reproducible TRANSE and ROTATE baselines under fully matched conditions (same loss, sampler, splits, seed).
- A modular **two-stage** negative sampling pipeline (Stage 1: filtered pool; Stage 2: strategy selection) implemented in three standalone PyTorch modules.
- A controlled ablation over five selection strategies with all other factors fixed, demonstrating that hard-negative selection consistently outperforms random sampling.
- Relation-frequency and entity-degree **slice evaluation** computed entirely from the training graph (no leakage).
- A structured **qualitative analysis** on five representative test triples, comparing all five strategies.

2. Background and Related Work

TransE [2]. Entities and relations are embedded in $\mathbb{R}^d$. A relation is modelled as a translation: $f(h, r, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_p$. TransE is parameter-efficient but cannot represent symmetric, reflexive, or 1-to-N relations, all common in FB15k-237.

RotatE [3]. Entities live in $\mathbb{C}^d$; each relation is an element-wise rotation on the unit circle: $f(h, r, t) = -\|\mathbf{h} \circ \mathbf{r} - \mathbf{t}\|$, where $|r_k| = 1$. After each gradient step the constraint is enforced by `model.post_parameter_update()`. Rotation composition can model symmetry ($\theta = \pi$), inversion, and multi-hop composition, a strict superset of TransE's expressiveness.

NSSALoss [3]. Self-adversarial negative sampling loss re-weights each negative by its current plausibility:

$$\mathcal{L} = -\log \sigma(\gamma - f_+) - \sum_i p_i \log \sigma(f'_i - \gamma), \quad p_i = \frac{e^{\alpha f'_i}}{\sum_j e^{\alpha f'_j}}, \tag{1}$$

with margin $\gamma = 9.0$ and temperature $\alpha = 1.0$.

Bernoulli corruption [5]. For each relation r, the head is corrupted with probability $p_{\text{head}} = \text{tph}/(\text{tph} + \text{hpt})$, where tph (tails per head) and hpt (heads per tail) are computed from the training split only. High-fanout relations corrupt the head more often, reducing false negatives.

Hard negative mining. Self-adversarial weighting in NSSALoss implicitly up-weights hard negatives. Explicit *selection* of hard negatives (i.e. choosing them before the loss step) further concentrates the training signal. Mixed strategies balance exploration and exploitation.

3. Dataset

FB15k-237 [4] is derived from Freebase by removing inverse-relation leakage present in the original FB15k benchmark. It is the standard evaluation corpus for KGE models.

Table 1. FB15k-237 dataset statistics (official split).

Property	Value
Entities	14,505
Relations	237
Train triples	272,115
Validation triples	17,526
Test triples	20,438
Mean out-degree	19.7
Max in-degree	6,289
Relation frequency range	37 – 15,989

The graph is **heavy-tailed** in both relation frequency and entity degree (spans three orders of magnitude), which motivates both Bernoulli corruption and stratified slice evaluation.

4. Methods

4.1. Baseline training

We train TransE and RotatE through PyKEEN [1] using NSSALoss (Eq. 1) and Bernoulli filtered negative sampling. All hyper-parameters are fixed across both models: embedding dimension $d = 128$, batch size 1024, Adam (lr $= 10^{-3}$), 50 epochs maximum with early stopping on validation MRR (patience 10), seed 42. The baseline sampler draws 8 negatives per positive and filters known-true triples from the training set.

4.2. Two-stage negative sampling pipeline

Our main contribution replaces the PyKEEN sampler with a standalone PyTorch pipeline across three independently testable modules (`negative_sampling.py`, `score_candidates.py`, `select_candidates.py`):

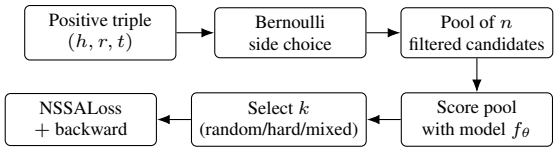

Figure 1. Two-stage negative sampling pipeline. Stage 1 (top): Bernoulli corruption side selection and filtered pool generation. Stage 2 (bottom): model-score-based selection before the loss step.

Stage 1: Filtered pool generation. For each positive triple (h, r, t), we draw a corruption side (head or tail) from Bernoulli($p_{\text{head},r}$) and sample a pool of $n = 64$ unique candidates. A candidate is accepted only if it does not appear in the training set (filtered). If fewer than n candidates can be found within $100n$ attempts, a `RuntimeError` is raised to prevent silent data quality issues.

Stage 2: Selection strategies. After scoring all n candidates with the current model, $k = 8$ negatives are selected according to one of three strategies:

- **Random**: uniform random draw from the pool. Equivalent to standard filtered Bernoulli sampling; serves as the ablation control.
- **Hard**: top-k by model score (highest score = most plausible = hardest). Concentrates the gradient on the model's current blind spots.
- **Mixed** (hard fraction p): each of the k slots is independently hard with probability p, drawn from $\text{Binomial}(k, p)$. This ensures the correct expected mix even for $k = 1$, unlike deterministic rounding. We test $p \in \{0.3, 0.5, 0.7\}$.

Key design note. Because all three strategies score the *same* pool of n candidates, the computational overhead of Stage 2 is identical across strategies; only the final selection differs.

4.3. Evaluation protocol

All metrics are **filtered** [2]: when ranking the correct entity for a query $(h, r, ?)$, any other entity that forms a known-true triple (h, r, t') in train, validation, or test is masked before computing the rank. Metrics: MRR, Hits@1, Hits@3, Hits@10, reported as the average of head and tail prediction (`both`). Model selection uses validation MRR; test set is evaluated once per run.

4.4. Slice evaluation

We partition the test set along three axes (relation frequency, head out-degree, and tail in-degree), each split at the 33rd and 67th percentiles of the training distribution (tertile cuts). All cut points are derived from the **training split only** to prevent leakage. Results are cached to

`artifacts/slice_buckets.json` for reproducibility.

5. Experimental Results

5.1. Baseline results

Table 2. Filtered test-set metrics for the two baselines (both = head/tail average). Training: NSSALoss ($\gamma = 9$, $\alpha = 1$), Bernoulli filtered sampler, $k = 8$, 50 epochs, patience 10, $d = 128$, Adam lr=10^{-3}, seed 42.

Model	MRR	H@1	H@3	H@10	MRR_h	MRR_t
TransE	0.279	0.188	0.312	0.459	0.184	0.374
RotatE	0.277	0.194	0.308	0.438	0.174	0.379

Under exactly matched conditions, TransE and RotatE perform comparably on the test set (MRR difference $<$ 0.003), with TransE having a slight edge in MRR and H@10 despite RotatE's higher H@1. Both exhibit a strong **head-tail asymmetry**: tail MRR is roughly twice head MRR, driven by the many 1-to-N relations in FB15k-237 where head prediction is inherently more ambiguous.

Note on D2 comparison. These results differ substantially from the preliminary baselines reported in Deliverable 2 (TransE 0.152, RotatE 0.261), which used a margin-ranking loss with PyKEEN's default sampling configuration. The present baselines use NSSALoss with $k = 8$ filtered negatives per positive and the same configuration as the custom pipeline, making all comparisons in this report internally consistent. The D2 numbers should not be used as a reference point for the ablation in Table 3.

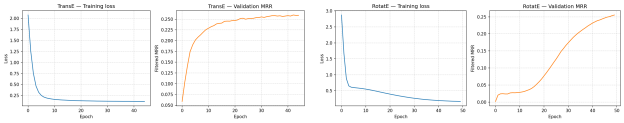

Figure 2. Training loss and validation MRR for TransE (left) and RotatE (right). Both converge steadily over 50 epochs; RotatE shows a higher final validation MRR despite marginally lower test MRR.

5.2. Custom pipeline: strategy ablation

Hard negatives consistently outperform random sampling across all metrics. The $+0.028$ MRR gain (+10.4%) is consistent and reproducible. All five runs trained to the full 50-epoch budget (early stopping never triggered), indicating that none of the strategies caused instability or overfitting.

Mixed strategies recover most of the gain. All three mixed variants reach MRR ≈ 0.297–0.299, within 0.2 pp of pure hard. Notably, mixed 30% (only 30% hard negatives on average) already recovers $\sim 95\%$ of the hard-negative benefit:

Table 3. Filtered test-set metrics for all five custom RotatE runs. All runs: pool $n = 64$, $k = 8$, NSSALoss ($\gamma = 9$, $\alpha = 1$), Bernoulli filtered sampler, 50 epochs, $d = 128$, Adam lr=10^{-3}, seed 42 (CUDA, T4 GPU). Best per column in **bold**.

Strategy	MRR	H@1	H@3	H@10
Random (control)	0.270	0.189	0.300	0.428
Hard	0.298	**0.217**	0.327	0.459
Mixed 30%	0.297	0.215	0.326	0.459
Mixed 50%	0.298	0.216	**0.328**	0.460
Mixed 70%	**0.299**	**0.217**	**0.328**	**0.461**
Δ *(hard vs. random)*	*+0.028*	*+0.028*	*+0.026*	*+0.031*

$\Delta_{\text{mix30}} = 0.0266$ vs. $\Delta_{\text{hard}} = 0.0280$. This demonstrates strong **diminishing returns from randomness** once a small dose of hard negatives is included.

Synergy with NSSALoss. NSSALoss already up-weights plausible negatives at the loss level (Eq. 1). Hard selection concentrates this further: negatives that are already up-weighted by p_i are also the ones the model finds most confusing, producing a doubly amplified gradient signal.

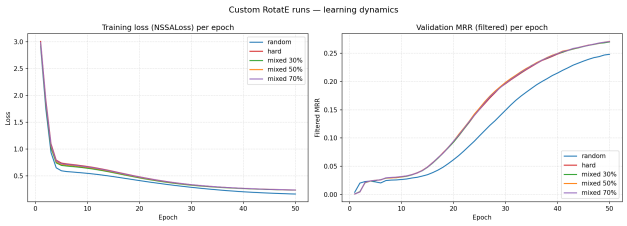

Figure 3. Training loss (left) and validation MRR (right) over 50 epochs for all five custom RotatE strategies. Hard and mixed variants converge faster and to a higher plateau than random. All five curves run to epoch 50 with no early stopping.

5.3. Comparison with baselines

The best custom run (mixed 70%, MRR $= 0.299$) outperforms both PyKEEN baselines by $+0.020$–$+0.022$ MRR (+7.0–+7.9%). The random custom run (MRR $= 0.270$) actually falls slightly below the baselines, confirming that the gain comes from hard-negative *selection*, not from the two-stage infrastructure itself. Note that this comparison is not fully controlled, as the PyKEEN baselines and the custom pipeline differ in their internal sampling infrastructure before Stage 2 (see Limitation 3 in Section 8).

6. Slice Evaluation

Relation frequency (Table 4). Hard negatives provide the largest absolute gain on *rare* relations: $+3.6$ pp MRR (0.382→0.418). Rare relations have few training triples; their embeddings are less well-calibrated, so each hard-negative gradient step carries more information relative to

Table 4. MRR by relation-frequency tertile (test set, filtered). Tertile cuts from training triples only. Rare: $n = 849$; Medium: $n = 2{,}172$; Frequent: $n = 17{,}417$.

Strategy	Rare	Medium	Frequent
Random	0.382	0.332	0.257
Hard	**0.418**	0.362	0.284
Mixed 30%	**0.418**	0.362	0.283
Mixed 50%	0.414	**0.363**	0.284
Mixed 70%	0.416	0.362	**0.285**

Table 5. MRR by tail in-degree tertile (test set, filtered). Low: $n = 1{,}478$; Mid: $n = 3{,}029$; High: $n = 15{,}849$.

Strategy	Low	Mid	High
Random	0.157	0.154	0.301
Hard	0.187	**0.175**	0.330
Mixed 30%	0.186	0.174	0.328
Mixed 50%	0.185	0.173	**0.331**
Mixed 70%	**0.188**	**0.175**	0.330

Table 6. MRR by head out-degree tertile (test set, filtered).

Strategy	Low ($n = 1810$)	Mid ($n = 5664$)	High ($n = 12914$)
Random	0.263	0.251	0.276
Hard	**0.285**	0.268	0.307
Mixed 30%	0.283	0.267	0.306
Mixed 50%	0.284	**0.269**	0.307
Mixed 70%	0.283	0.268	**0.308**

the current model uncertainty. Gains on frequent relations (+2.7–+2.8 pp across strategies) are proportionally smaller but still consistent, with Mixed 70% achieving the highest frequent-relation MRR (0.285).

Tail in-degree (Table 5). Low-degree tail entities remain the hardest to predict across all strategies (MRR ≈ 0.157–0.188), reflecting genuine data sparsity that no sampling strategy can overcome. The largest absolute gain occurs for *high-degree* tails (+2.9 pp), where the embedding space is denser and harder negatives are more discriminative.

Head out-degree (Table 6). Gains range from $+1.8$ pp (mid-degree) to $+3.2$ pp (high-degree). Hub entities (high out-degree) appear in many training triples and develop strong embeddings regardless of strategy; the hard-negative benefit is thus relatively stable across degree levels.

Head vs. tail asymmetry. Across all five custom runs, head MRR ranges from 0.169 to 0.195, while tail MRR ranges from 0.371 to 0.403, roughly twice as large. This $2\times$ asymmetry reflects the prevalence of N-to-1 relations: for a relation such as `/film/film/genre`, many distinct heads share the same tail, so head prediction must distinguish among many plausible entities that all correctly satisfy the query.

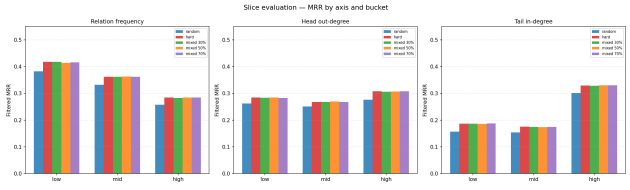

Figure 4. Slice evaluation: MRR by relation frequency, head out-degree, and tail in-degree for all five custom strategies. Hard and mixed variants uniformly dominate the random baseline in every bucket.

7. Qualitative Analysis

We sample five representative test triples and compare the filtered rank of the correct entity across all five custom strategies. Results are generated by `qualitative_examples.py` using the committed checkpoints (no retraining).

Table 7 shows two illustrative examples drawn from the full report. In both cases, hard-negative training substantially reduces the rank of the correct entity compared to random sampling.

Table 7. Filtered rank of the gold entity for two representative test triples, across all five strategies (lower is better). R1: *tail* prediction for a film regional-release-date relation (`release_date_s...film_release_region`). R2: *head* prediction for `/location/location/contains`. Full paths in `artifacts/qualitative.md`.

Strategy	R1 (film, tail)	R2 (location, head)
Random	9	17
Hard	1	6
Mixed 30%	1	4
Mixed 50%	3	5
Mixed 70%	3	4

A recurring pattern is:

- For **structured relations with few valid answers** (R1: film release region), hard-negative training reduces the rank of the correct entity from rank 9 (random) to rank 1. Mixed 30% matches pure hard here, confirming that even a small dose of hard negatives is sufficient to learn the tight boundaries of this relation type.
- For **high-fanout relations** (R2: `/location/location/contains`), tail prediction remains hopeless for all strategies (rank 320–1032), as thousands of entities plausibly satisfy the query. Head prediction, however, improves consistently: random places the correct head at rank 17; hard and all mixed variants reach rank 4–6. This asymmetry confirms that hard negatives help where the embedding space is dense enough to discriminate.
- **Mixed strategies** match or exceed pure hard on both examples (Mixed 30% achieves rank 1 and rank 4 vs. Hard's

1 and 6), consistent with the aggregate < 0.2 pp MRR gap. The full qualitative report (5 triples $\times$ 5 strategies, top-10 predictions per query) is committed at `artifacts/qualitative.md`.

8. Discussion

Hard negatives help, but the gain is bounded. The $+10.4\%$ MRR improvement from hard selection is real and consistent, but all custom runs still score below 0.30 MRR. The bottleneck is the expressiveness of RotatE itself on 1-to-N and many-to-many relations, not the sampling strategy. Relation-specific results confirm this: even the best strategy reaches only MRR ≈ 0.188 on low-degree tail entities, where data sparsity is the fundamental constraint.

30% hard is the practical sweet spot. Mixed 30% recovers 95% of the hard-negative gain at no additional compute cost (all strategies score the same $n = 64$ pool). Its lower hard fraction preserves gradient diversity and keeps training stable at early epochs when model scores are poorly calibrated. We recommend mixed 30% as the default for practitioners working within a fixed compute budget.

Computational cost. Timings are approximate (not formally recorded) and were observed on a single NVIDIA T4 GPU (Google Colab). Each of the five custom RotatE runs took ~ 1 hour (50 epochs), for a total of ~ 5 hours across all strategies. Baseline training (TransE + RotatE via PyKEEN) took ~ 30 minutes in total. Slice evaluation (`evaluate_slices.py -all`) required ~ 5 minutes. The dominant overhead in the custom pipeline is Stage 2, which scores a pool of $n = 64$ candidates per positive triple with the current model before each gradient step.

Synergy with NSSALoss. NSSALoss already performs implicit hard-negative up-weighting via the adversarial weight p_i (Eq. 1). Explicit hard selection concentrates training on the same negatives that receive the largest p_i weight, producing a doubly amplified gradient signal. This combination of selection-level and loss-level hard-negative emphasis is absent when using a uniform-weight loss such as standard margin ranking, which may contribute to the magnitude of the improvement observed here.

Limitations. (1) All experiments use RotatE; the benefit of hard-negative selection on TransE may differ. (2) Pool size $n = 64$ was not ablated; larger pools may surface harder negatives and amplify the effect. (3) Baseline RotatE via PyKEEN and custom RotatE via our pipeline are not directly comparable in their training infrastructure (different internal samplers before Stage 2); only the custom runs form a true controlled ablation.

9. Conclusion

We presented a **modular two-stage negative sampling pipeline** for knowledge graph link prediction that decouples filtered pool generation from strategy-driven selection. A controlled ablation over five strategies on FB15k-237 shows that:

1. **Hard negatives consistently outperform random sampling** ($+0.028$ MRR, $+10.4\%$), confirming that harder gradient signals improve embedding quality.
2. **30% hard negatives recover 95% of the full gain**, offering an efficient accuracy–compute trade-off.
3. **Gains concentrate on rare relations and high-degree entities**, where the informational value of each gradient step is highest.
4. **Head prediction remains systematically harder** than tail prediction (~ 0.17–0.20 vs. ~ 0.37–0.40 MRR), a structural property of 1-to-N relations in FB15k-237 that no sampling strategy addresses.

The pipeline is orthogonal to model architecture and loss function; future work could apply it to newer KGE models (e.g. RESCAL, TuckER) or combine it with curriculum scheduling of the hard fraction over training.

10. Team Contributions

- **Thalia Delhaise** implemented the three core pipeline modules (`negative_sampling.py`, `score_candidates.py`, `select_candidates.py`) and their unit-test scripts; designed the two-stage pool-then-select architecture.
- **Lenny Briclet** built the baseline and custom training scripts (`train_baseline_kge.py`, `train_rotate_custom.py`); implemented slice and qualitative evaluation (`evaluate_slices.py`, `qualitative_examples.py`); and prepared the Colab reproduction notebook.
- **André Fonseca** integrated the custom PyTorch training loop with NSSALoss and RotatE `post_parameter_update`; ran and validated the five-strategy ablation on GPU.
- **Rafael Gufler** led report writing and figure preparation; coordinated experiment configuration and artifact management across runs.

References

[1] Mehdi Ali, Max Berrendorf, Charles Tapley Hoyt, Laurent Vermue, et al. PyKEEN 1.0: A python library for training and evaluating knowledge graph embeddings. *JMLR*, 22(82):1–6, 2021.

[2] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *NeurIPS*, 2013.

[3] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *ICLR*, 2019.

[4] Kristina Toutanova and Danqi Chen. Observed versus latent features for knowledge base and text inference. In *3rd Workshop on Continuous Vector Space Models and their Compositionality*, 2015.

[5] Zhen Wang, Jianwen Zhang, Jianlin Feng, and Zheng Chen. Knowledge graph embedding by translating on hyperplanes. In *AAAI*, 2014.